From minigame demo to playable game — a step-by-step tutorial

You have a working, tested client-side minigame (`Smash Garden`). What you *don't* have yet is a **game**: a place in a stage, a reason to play, a server that starts and judges rounds, rewards, and progression.

This tutorial takes you there in 8 steps. Each step ends with a concrete **checkpoint** you can verify in-game (or in the headless test).

> **The mental model that never changes: Lua = client** (UI, input, audio, tweens) · **Node Graphs = server** (world, rules, rewards, persistence) · **Signals = the wire** between them. Every step below is either client work, server work, or extending the *contract* between them.

Reference material: `README.md` (setup, hierarchy, signal contract), `../MILILUA_ROADMAP.md` (broader learning path), and the code itself — every script has a `[PATTERN TOUR]` header.

Step 0 — What you have, what's missing

Layer	Status
Client game loop (states, countdown, scoring, pause, results)	✅ built + tested
Input parity (keyboard + controller)	✅ built
Localization plumbing (all strings via `game.GetText`)	✅ built (en + ja)
Teardown discipline (survives loading screens)	✅ built
A stage that hosts it	❌ this tutorial, Step 2
Server: start/abort/judge rounds	❌ Step 3–4
Persistence + progression	❌ Step 5
More content (waves, new targets, fever)	❌ Step 6
Feel/polish + QA	❌ Steps 7–8

Step 1 — Get it running in the editor (30–60 min)

Nothing else in this tutorial is real until the demo runs inside the game.

1. Copy `lua/minigame/*.lua` into the game's **`external_lua_file`** directory (script paths in this project assume the `minigame/` prefix).
2. Follow `README.md` → *Editor setup*: build the control hierarchy, register the three templates, map the scripts, add the Text Mapping entries (en + ja), and attach the Custom Variable component.

3. Add the **debug HUD** (it exists exactly for this step):
    - Add a `DebugText` TextWindow under `Hud` (top-left, small font).
    - Map `debug_hud.lua` onto it, script var `Enabled=1`.
4. **Test Play.** Expect, in order:
    - Console: `[SmashGarden] OnInit — round=30s`, `[SmashGarden] ready.`
    - DebugText: `IDLE score=0 combo=0 ... targets=0` (refreshing 4×/s)
    - Press **F** → 3·2·1·GO → targets pop in → click them → score rises
    - **Tab** → pause overlay, timer freezes → Resume → timer resumes
    - Let the round run out → results slide in, replay works

Checkpoint 1: full round completed with keyboard. Repeat with a controller (F ≈ whatever the interact key maps to; ESC ≈ menu-back).

> Anything red in the console at boot is an *editor* problem (missing control, missing script var, unmapped text key) — not a Lua problem. The code fails loudly on purpose.

Step 2 — Give the game a home (editor work)

A minigame floating in an empty test stage isn't a game. Build the place it lives:

1. **An arena**: a garden/dojo/arcade corner — whatever fits your world. The play field is a 3×3 slot grid on screen; the world just needs to feel like a place. Standalone mini-game stages are a normal thing in Miliastra — the whole stage can *be* the arena.
2. **An entry point**: a trigger volume or interactable at the entrance. The player walks in → the server decides a round begins (Step 3).
3. **UI root per stage**: `SmashGarden` and its children go into *this* stage's Interface Layout. Controls die on loading screens by design — when the player teleports in, `OnStart` rebuilds everything. Don't fight it; the code already handles it.
4. **Set** `AllowManualStart=0` on the director's script mapping. The F key becomes inert; only the server can start rounds now. (Keep it `1` while you're still building the server graph — both paths work simultaneously, so you can flip it the moment the graph is ready.)

Checkpoint 2: Test Play, walk into the arena, press F — nothing happens. The client is now *ready to be told what to do*.

Step 3 — The server starts the game (node graph)

This is your first real signal contract extension. In the stage's node graph editor (see the wiki for the exact node palette — names vary by version, look for *trigger*, *timer*, and *Send Client Scripted Signal*):

```
[Player enters arena trigger]
        |
        ▼
Send Client Scripted Signal "StartRound"   params: [1] int = 30
```

On the client nothing changes — `director.lua` already listens:

```
script:RegisterServerSignalHandler("StartRound", function(_, params)
    startRound(params and params[1])   -- params[1] is the duration override
end)
```

Remember the two iron rules (README → *Node graph wiring*):

- Params are matched **by index**, in the exact order of the **Server Signal Explorer**. Sender and receiver must agree on `[1]=duration, [2]=..., [3]=...`.
- Server→client signals have **~100 ms minimum latency**. That's fine for "start/stop/tune a round" and fatal for anything frame-critical — all round *timing* stays client-side for this reason.

Checkpoint 3: Test Play with the node graph active. Walk into the trigger → countdown begins without F. Walk out (or teleport) mid-round → add a second graph path for that in Step 4.

Step 4 — The full round lifecycle graph

A round that only *starts* isn't a game loop. Complete the contract — all five signals already exist client-side; you're wiring the other end:

Signal	Direction	Your graph does
`StartRound [1]int`	→ client	Trigger entered / "Play again" NPC option
`AbortRound`	→ client	Trigger exited, player died, stage ending, teleport
`SetGoldenRate [1]int`	→ client	Stage Timer node: 0% at start, e.g. 40% halfway — a difficulty spike defined in *data*, not code
`RoundComplete int,int,int,int`	← client	**Judge the round** (below)
`SubmitHighScore int`	← client	Compare with `BestScore` Custom Variable; set it if higher

Judging: the client reports, the server decides

`RoundComplete` arrives as `score, hits, misses, bestCombo`. The client *could* be lying (modified Lua, lucky bugs). Use it for rewards, but sanity-check it first. Cheap plausibility checks:

- `score ≤ hits × 50` — the max possible per-hit value (5× multiplier at ×5 combo tier × base 10). Anything higher is impossible.

- `hits ≤ roundDuration × 3` — at most `MAX_ACTIVE=3` targets alive and the spawn ramp never goes below 0.25 s; a generous ceiling.
- `bestCombo ≤ hits` — combo can't exceed hits.

Fail the check → ignore the reward, log it, send `AbortRound`. This is the same "trust, but verify" pattern the director already applies to the target-reported multiplier — just one layer up.

The quit path

`AbortRound` matters more than it looks: `endRound()` **unfreezes level time** and force-closes the pause menu even mid-pause. Wire it to every "this stage is over for this player" path (exit trigger, death, GM kick) and the client can never wedge itself in a paused or running state.

Checkpoint 4: enter → play → leave mid-round → re-enter. Every transition starts or ends a clean round; the server graph received a `RoundComplete` each time (log it server-side while testing).

Step 5 — Persistence and meta progression

Right now `BestScore` lives in a Player Custom Variable, seeded in `OnStart` and kept fresh by a change handler. Extend the pattern — every new persisted value is three small moves:

1. **Component**: add the Custom Variable to the Player entity in the editor (name it explicitly, e.g. `SmashTotals`).
2. **Read**: seed in the director's `OnStart` (`game.GetGlobalCustomVariableValue`) + register a changed handler.
3. **Write**: never write from Lua — send a signal (`SubmitSmashTotals`) and let the graph's Set Custom Variable node do it. The server owns state; the client owns presentation.

Ideas that cost almost nothing and make it feel like a real game:

- **Totals**: rounds played, total whacks, best golden streak → show on the results panel next to the run stats.
- **"New Best!" banner** — pure client polish on data you already have. In the editor add a `NewBestText` TextWindow under `Results` (starts hidden), then in `results_panel.lua`:

  ```
  -- in Show(), after the other text writes:
  ui.newBestText:SetVisible(score > 0 and score >= bestScore)
  ```

  (`bestScore` arrives *before* the new record overwrites the custom variable in the general case — if your persistence lags a frame, the server echoes the updated variable back and the changed handler refreshes `bestScore` anyway. One owner per piece of state.)
- **Currency/quest hooks**: `RoundComplete` is also the hook your world's economy graph listens to — "win 3 rounds" quests, shop currency, titles.

Checkpoint 5: finish a record round → New Best banner; check the Custom Variable actually changed (server-side inspection); relog/teleport and confirm it persisted.

Step 6 — Content: make it more than one game

All of these are **data and templates first, code second** — the golden target already proved the pattern.

6a. Wave / difficulty mode — zero code

The `StartRound` duration override is a difficulty dial. Graph: stage timer sends `StartRound(20)` → on `RoundComplete`, judge sends `StartRound(15)` → 10 → ... until a threshold fails. The client already clamps durations to sane bounds (`5–300 s`).

6b. New target types — templates, not branches

Duplicate a target template, change its script vars:

Template	Vars	Behavior
Golden	`PointsMultiplier=5`	exists
Silver	`PointsMultiplier=2`, lifetime default	steady value
Bomb	`PointsMultiplier=0`, `Penalty=1`	breaks combo (needs the 10-line patch below)

Bomb patch — `target.lua` reports its penalty flag; the director applies it. In `target.lua`:

```
local PENALTY = false
function OnInit()
    POINTS_MULTIPLIER = tonumber(script:GetParam("PointsMultiplier")) or 1
    PENALTY = tonumber(script:GetParam("Penalty")) == 1
end
-- in onClick, after dir resolves:
dir:Invoke("OnTargetClicked", self_.id, POINTS_MULTIPLIER, PENALTY)
```

In `director.lua` `OnTargetClicked(controlId, multiplier, penalty)`:

```
if penalty then
    combo = 0
    playSfx(MISS_SFX_ID)
    spawnPopup(x, y, game.GetText("minigame.bomb"), COLOR_TIMER_LOW)
    refreshHud()
    return
end
```

(Add a `minigame.bomb` text entry — remember, no hardcoded strings.)

6c. Fever mode — the juice hook

At `combo == 10` (and every 10 after): double points until the combo breaks, plus a screen flash. The API has `ClientUIFullscreenAnimationControl` for exactly this (per the docs — add the control, drive it from the director when the fever flag flips). Points change is two lines in `OnTargetClicked`:

```
local fever = combo >= 10 and (combo % 10 == 0 or feverActive)
```

Keep the fever *state* in the director (one owner per piece of state), and show it in the HUD with a tweened color — `fontColor` on the combo text is `[Tweenable]`.

Checkpoint 6: three templates spawn (rate-tune each via `SetGoldenRate`-style signals or per-template spawn weights); a bomb breaks the combo; fever triggers at 10 and pays double.

Step 7 — Feel and polish pass

The difference between "works" and "plays well" is almost entirely here, and it's all client-side — your happy place:

1. **Audio layering**: pitch/layer the hit SFX by combo tier instead of one flat clip (`PlayAudio2D` per tier via script vars), a stinger on round start, crowd noise under fever.
2. **Controller + key hints**: the API has `ClientUIKeyHintControl` — show "Start: interact" near the arena entrance, and make hints device-aware (`game.GetDevice()`), like the pause menu already does.
3. **Localization pass**: run the game in each supported language (change client language in test play). Every new string from Step 6 needs its ja (and any other) column **now**, not later.
4. **Numbers pass**: play 10 rounds and *feel* the curve. Then change **data, not code**: `RoundDuration`, spawn-ramp constants at the top of `director.lua`, `PointsMultiplier`s, `SetGoldenRate` timeline in the graph. If you can't tune it without editing logic, pull the number into a script var first.
5. **Performance, only if needed**: the demo instantiates/destroys targets and popups. At `MAX_ACTIVE=3` that's free. If you crank spawn rates 10×, switch to **pooling**: keep the instantiated controls in their slots and recycle them (hide + reset scale) instead of destroying — the slot grid makes this a natural refactor.

Checkpoint 7: a colleague plays with sound on and a controller, in a second language, and says "again" without being asked.

Step 8 — QA, then ship

Client matrix (mostly re-run the headless suite + one manual pass)

- [ ] `lua5.4 test/run_test.lua` still green after every Lua change — the suite covers scoring, pause, abort, replay, teardown, and tween-lock discipline. Extend it when you add signals.
- [ ] Teleport/reconnect mid-round, mid-pause, mid-countdown → next stage is never frozen, no errors, `OnDestroy` cleaned up.
- [ ] Enter arena → *immediately* leave → server sent `AbortRound` before `StartRound` even landed (~100 ms latency race). Client handles it (it does — abort only acts in `PLAYING`, and a late `StartRound` after an abort is the one race to test explicitly).
- [ ] Controller-only run: pause, resume, quit, replay, results focus.
- [ ] Every language you ship.

- [ ] `Enabled=0` on the debug HUD, `AllowManualStart=0` on the director — **debug affordances off for the release build**.

Server matrix

- [ ] High score only ever increases (compare before set).
- [ ] Plausibility checks reject an obviously forged `RoundComplete`.
- [ ] Rewards granted exactly once per `RoundComplete` (replay the signal if a player disconnects mid-judgment — design for idempotency).

Ship

- [ ] Publish, watch the first 10 minutes of real play hardest — that's where people bounce off.
- [ ] Iterate from actual telemetry: your `RoundComplete` params *are* your analytics stream.

Final checkpoint: a stranger finds your stage, plays a round, beats their score, quits, comes back later — and their best score is still there. Congratulations: that's a game.

Appendix — evolving the signal contract

The contract (`README.md` table) is versioned in *your* discipline, not in code:

1. **Never reorder or repurpose params.** Add new signals or append params; index-matching makes silent breakage the failure mode.
2. Keep the `---@class` annotations (see the top of `director.lua`) in sync — they're your contract's documentation inside the code.
3. When you extend it, extend the test: fake both ends (`H.sendSignal` / `H.sentSignals`) and assert the new params in order.